

Forge for Parts Town

Forge is a design-system engine. It keeps the design system as data — what a Part *is*, what it can do (be priced, superseded, fit-checked), which screens it appears on, and the tokens that style it — and from that one source generates screens, charts, themes and code, then machine-checks what it made. It holds the *definition* of your catalog, not the catalog.

WHY IT MATTERS HERE One definition of a Part can serve the JSP storefront today, Angular later, and iOS / Android — and your brand, expressed as tokens, outlives the re-platform.

5	587	5	8	13	4
PARTS-CATALOG OBJECTS MODELED	OF YOUR FIGMA TOKENS IN SCOPE	PLATFORMS FROM ONE TOKEN SOURCE	SCREEN CONTEXTS	CHART TYPES	CERTIFICATION CHECKS

What Forge is AND WHAT IT IS NOT

Most design systems are a library of components plus a document on how to use them. Forge keeps the system as *data* — objects, their traits, the contexts they appear in, the tokens that style it all — and generates from it: screens, charts and dashboards, theme CSS, native token files for iOS and Android, HTML or framework code. Generated charts carry a content fingerprint and come back with a machine-checked verdict on accessibility, determinism, contrast and accuracy: output you can *prove* things about, not just produce faster.

HOW IT IS OPERATED

By software agents, not a point-and-click app: an AI assistant under our direction calls 25 typed tools to describe, compose, validate, render, certify and export. Forge runs with us; you would receive its outputs in a repository you own. Nobody at Parts Town installs Forge.

WHAT FORGE IS NOT

- Not a hosted app or a login for your team (a hosted endpoint is a later option).
- Not a replacement for Figma — it reads from it. Not a CMS or PIM — definitions, not catalog data.
- Not an Angular generator today (HTML, React, Vue): for your storefront the near-term deliverable is tokens, theme CSS and HTML references; an Angular output is scoped work if the re-platform is real and near.

The vocabulary

SIX TERMS THAT CARRY THE WHOLE SYSTEM

Trait a capability an object can have

"Can be priced", "can be superseded", "has a lifecycle" — packaged with everything it needs: data fields, settings, the UI pieces it adds to each kind of screen, the tokens that style them, accessibility notes. Traits, not widgets, are the unit of reuse.

Example. **Stateful** adds a status and a lifecycle, and contributes a status badge to lists, a status timeline to detail pages and a status selector to forms — on every object that uses it.

PARTS TOWN Already modeled: **Identifiable** (house SKU vs. manufacturer part number vs. OEM / UPC / GTIN), **Supersedable** (replacement chains that resolve to something orderable), **Fitmentable** ("does this part fit this unit?"), **AttributeBearing** (typed spec facets for filtering).

Context the kind of screen

List, detail, form, timeline, card, inline, chart, dashboard. Traits declare what they contribute to each context and where; Forge assembles the screen in a fixed, deterministic order — "a Part in a results list" and "a Part on its detail page" come from one definition, not two hand-built pages. In Forge's component library every context shares one six-region contract, rendered as the landmarks screen readers navigate by and collapsing to one column below tablet width.

Object a thing your business talks about

A Part, a Manufacturer — declared as the list of traits it has, each with its settings, plus the few fields truly its own. Forge merges the traits (warning on any field two traits both claim) and uses that one record to build screens, charts and TypeScript types — e.g. **Product** is eight traits plus its own SKU and inventory status.

PARTS TOWN Already modeled: **Part**, **Equipment Model**, **Manufacturer**, **Fitment** (the qualified part↔equipment edge: position, quantity, serial range, verification source) and **Cross-Reference** (cross-brand interchange with grade, direction and confidence).

Token & Brand every visual value, once

Colours, type, spacing, motion and breakpoints are tokens in the W3C Design Tokens standard (DTCG). One build compiles them to CSS variables, a Tailwind token map, TypeScript, Swift and Kotlin (per-brand native sets in progress). Tokens are layered from raw values up to named roles — "primary", "surface", "text on accent" — and a brand is a set of values for those roles, so swapping Parts Town in touches no component. Brand and theme (light / dark / high-contrast) are two attributes on the page; the build refuses to run if two sources silently disagree on a token.

Generate from intent to artifact

Name an object and a context, or give a short intent phrase the composer matches to a layout ("an order-history list with filters"). Forge composes a validated screen description from its catalog of about a hundred components, renders it to self-contained HTML, and can emit React or Vue source against its own component library. Hand it data and it returns a chart spec (13 types) with, on request, an accessible data table and narrative from the same data.

Certify prove it, don't assume it

Generated charts come back with a verdict across four checks: **accessibility equivalence** (does the text alternative say what the chart shows — 16 rules), **determinism** (rebuild twice, compare bytes, hash), **contrast** (WCAG 3:1 plus colour-vision-deficiency distinguishability, on the light theme) and **accuracy** (structural distortions such as a truncated bar baseline — 10 rules). The full certificate covers the five core chart types today; the other eight report each check separately. Screens can be checked against 18 token-contrast pairs on request; Forge's component library is axe-audited in CI.

How it works MODEL THE DOMAIN, SEED THE BRAND, THEN ONE PASS PER REQUEST

01 Describe Our operator (an AI assistant under our direction) names an object and a context, or a short intent.	02 Compose Forge picks a layout, fills it from the catalog via the object's traits, flags any slot it was unsure about.	03 Validate Structural check; the 18 token-contrast rules can run here too.	04 Render Self-contained HTML; with a brand applied, light / dark / high-contrast are page attributes.	05 Certify Five core chart types get the four-check certificate and a content hash; the other eight, per-check verdicts.	06 Export HTML, plus React / Vue source; theme CSS; token files for web, iOS, Android; one-file dashboards.
--	---	---	--	--	---

Under the hood: a local server speaking the Model Context Protocol (the standard AI assistants use to call tools), with roles, rate limits and an explicit opt-in before anything is written. Intent phrases are matched by rules and synonyms, not a language model — same input, same bytes, same hash; the assistant sits outside Forge and calls it.

What makes it different

VS. A COMPONENT LIBRARY OR A FIGMA-TO-CODE TOOL

It composes from meaning, not pixels The unit of reuse is a capability — priceable, stateful, supersedable, fit-checkable — carrying data contract, UI pieces, tokens and accessibility notes together. Adding the lifecycle trait to a new object is one line that lights up its status badge, timeline and selector in every context.	One definition, every context Every view of an object is assembled from one declaration, each UI piece traceable to the trait that contributed it. That traceability is what makes the output checkable rather than merely plausible.	It certifies what it generates Generated charts come back with a machine-checked verdict rather than a promise; among the design-system tools and hosted endpoints we have surveyed, none returns one. Today the certificate covers charts; screens get contrast rules and automated audits. The same bar applies to Forge's own build: the token build fails on a silent collision, 108 generated brand-contrast rules run in CI, and the Swift and Kotlin files are compiled with the real toolchains before each release.
---	---	--

What Forge could do for Parts Town

STATUS AS OF AUGUST 2026

READY exists today; needs only your brand	ENGINE WORK we build it; scoped
YOUR INPUT needs a decision from you	

CAPABILITY	WHAT YOU WOULD GET	STATUS
Your brand as tokens	Your colours, type, spacing and breakpoints as DTCG tokens → CSS variables, a Tailwind token map, TypeScript, Swift and Kotlin, in light / dark / high-contrast; the same CSS variables serve JSP today and Angular later. A first pass over a late-July export of your Figma file is already converted (713 entries; 587 in scope; 225 base tokens plus 84 mode values) with a coverage report itemising every collision, missing value and rename. It surfaced questions to settle together: two "Parts Town Red" values, Museo Slab and Effra under the same style names, four empty tokens, some third-party kit tokens mixed in.	ENGINE WORK YOUR INPUT Seed the brand, add web-font loading; you pick canonical values and supply licensed web-font files. Works from Figma exports; a live Figma connection is new work and depends on your plan tier.
Parts-catalog screens from a domain model	The five objects and four traits above, composed into list, detail, form and timeline screens from one model — replacement banner, supersession timeline, compatibility panel, fit verifier, specification table, attribute facets, identifier panel. Also a shared spec across web and native apps.	MODEL DRAFTED ENGINE WORK Definitions exist; not yet loaded into the engine, and its 15 parts-specific components are still to be built.

CAPABILITY	WHAT YOU WOULD GET	STATUS
Accessibility guardrails built in	In Forge's component library: landmarked layouts, toolbar roles and labels, an opt-in focus-trapped side-panel drawer, light / dark / high-contrast themes, phone and tablet test runs. For any generated screen: a token-contrast check (18 WCAG pairs) you can switch on. Plus axe checks on library screens in our CI, and certified charts. Relevant to ADA / WCAG 2.1 AA exposure on a US storefront.	READY Automated guardrails on what Forge makes — not a WCAG audit of your storefront. Chart certificates exist; page certificates do not.
Certified analytics & dashboards	For reporting (merchandising, fulfilment, supersession coverage) or customer-facing order analytics: 13 chart types, each with an accessible data table and narrative. Bar, line, area, scatter and heatmap get the full four-check certificate; the other eight (treemap, sunburst, sankey, network, chord, three map types) get each check separately and are labelled "uncertified" on purpose until their accessibility check is enforced. Dashboards export as one self-contained HTML file — the five core types as inline SVG identical on every rerun, the others as described placeholders today.	READY (five core types) ENGINE WORK (the eight) Certificates are brand-independent — they grade the colours baked into the chart; dashboard chrome takes your brand once seeded.
Native iOS / Android tokens	Swift and Kotlin token files from the same source as the web, compiled with the real toolchains before each release.	ENGINE WORK One brand and theme today (colour, size, spacing, duration); font families and motion curves not yet; per-brand native sets still to build.

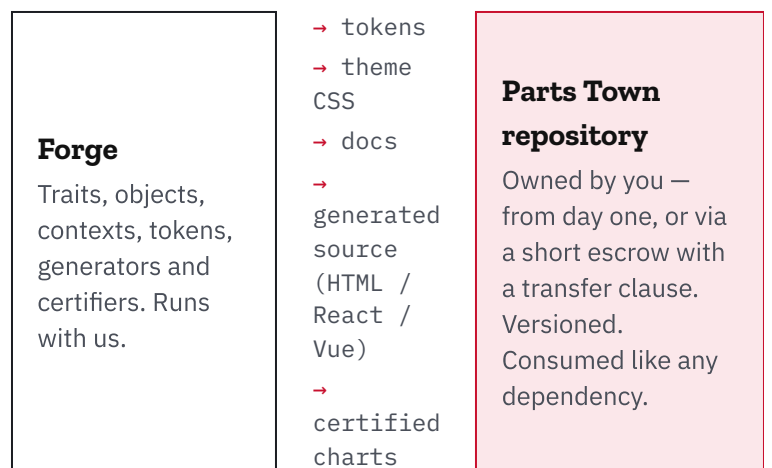
How we would deliver it

FORGE IS THE FACTORY; PARTS TOWN OWNS THE SHELF

Parts Town would be developed as a brand *inside* Forge, so every generator, check and export applies to it. You would receive a repository you own: DTCG tokens, built theme CSS and Tailwind mapping, documentation and a static component-reference site you can host anywhere, generated source where you want it, and certified charts with a fingerprint you can recompute. We do not fork Forge for you, and you never run it — the engine stays with us and the products travel. That keeps your exit cheap: if the system is ever developed elsewhere, the tokens and docs are already yours.

A first slice

1. Seed your brand from a fresh Figma export; settle the two reds and two typefaces with you.
2. Emit CSS variables, the Tailwind map, Swift and Kotlin files into your repository.



- ↳ Web storefront (JSP today, Angular later)
- ↳ iOS
- ↳ Android
- ↳ Design (Figma)

- 3. Render the Part list and detail pages in your brand as a reference.
- 1. Hand over with the component-reference site and governance notes.

What we would want to settle with you

EACH ANSWER CHANGES THE PLAN

- 1. Scope: tokens, theme and documentation only, or generated components too? The largest sizing variable.
- 2. Which Parts Town Red and which type family are canonical (your file carries both), and is dark mode real or light-only? A fresh export re-bases everything.
- 3. Your Figma plan tier — it decides which ingestion path we build: a live connection or a repeatable export.
- 1. GitHub and package-registry policy — npm package, tarball, or a source drop.
- 5. Is the Angular storefront migration real and near? It decides tokens-only versus components now.
- 6. Who owns the repository and package scope on day one — you, or escrow with a transfer clause — and who owns the system on your side day to day?
- 7. iOS, Android or mobile web first — and do your apps consume design tokens today?
- 8. Accessibility-compliance exposure or appetite — it decides whether certified output is a line item or a quality bar.

Figures reflect the Forge repository in August 2026: 41 traits · 11 object definitions (+ 5 parts-catalog objects modeled for you) · ~100 catalogued components · 8 screen contexts · 13 chart types · 5 token platforms · 25 agent tools.

Prepared for Parts Town by [name · studio · email]